ARIMA aplicado a COLCAP y Bitcoin: ¿hay estructura que explotar?

Andrés Londoño

1. Descripción de las series de tiempo

COLCAP. El COLCAP es el índice de capitalización de la Bolsa de Valores de Colombia (BVC), calculado a partir de las 20 acciones más líquidas del mercado colombiano, ponderadas por su capitalización bursátil ajustada. Se trabaja con el cierre mensual del índice entre enero de 2021 y agosto de 2026 (68 observaciones), tomado de la serie histórica diaria publicada por el Banco de la República con fuente BVC.

Bitcoin (BTC-USD). Bitcoin es la principal criptomoneda por capitalización de mercado, cotizada las 24 horas los 7 días de la semana contra el dólar estadounidense. Se trabaja con el precio de cierre mensual entre enero de 2021 y agosto de 2026 (68 observaciones), tomado de Yahoo Finance.

Se anuncian y muestran las dos series originales, en niveles:

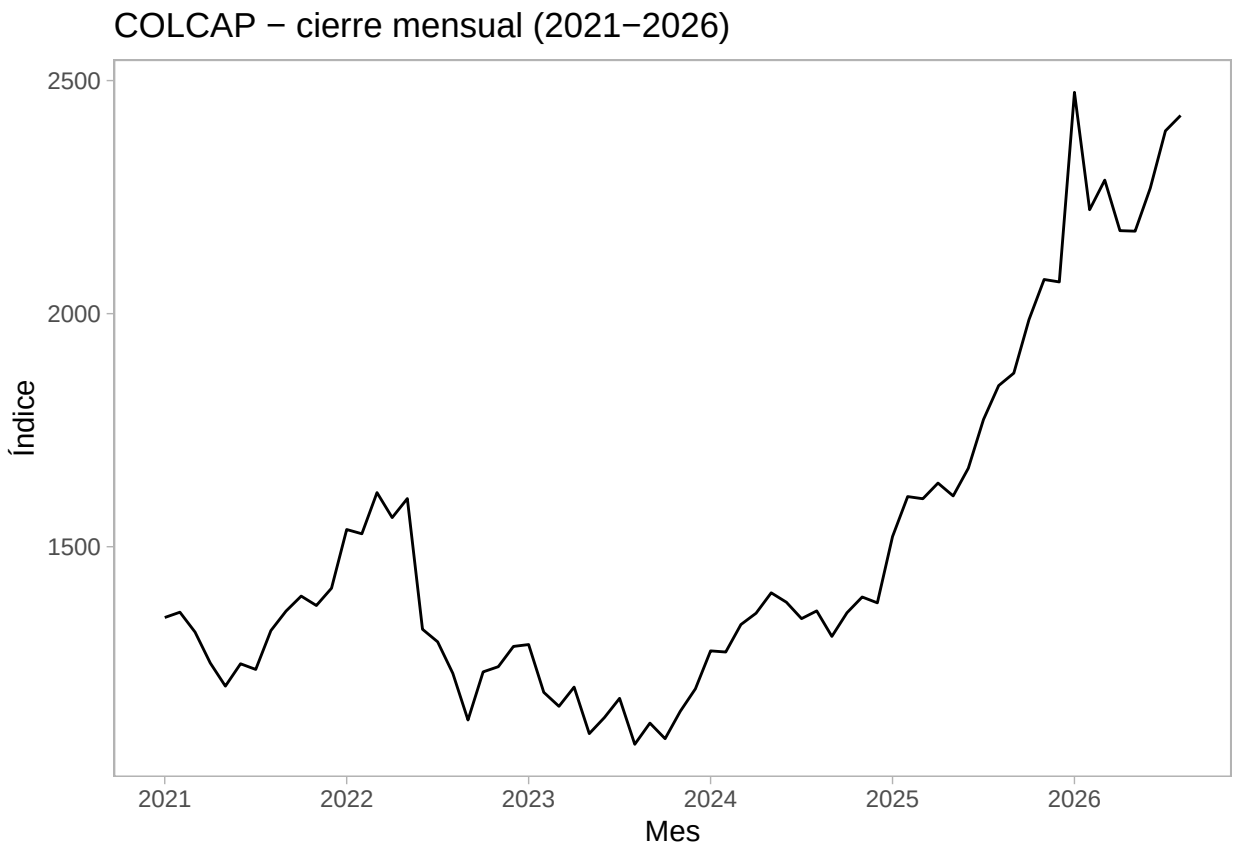
```
if (!require("pacman")) install.packages("pacman")
pacman::p_load(tidyverse, xts, tseries, forecast, lmtest, quantmod, aTSA, scales, FinTS)
options(scipen = 999)

source(file = "Setup/Pruebas residuales ARIMA.R")

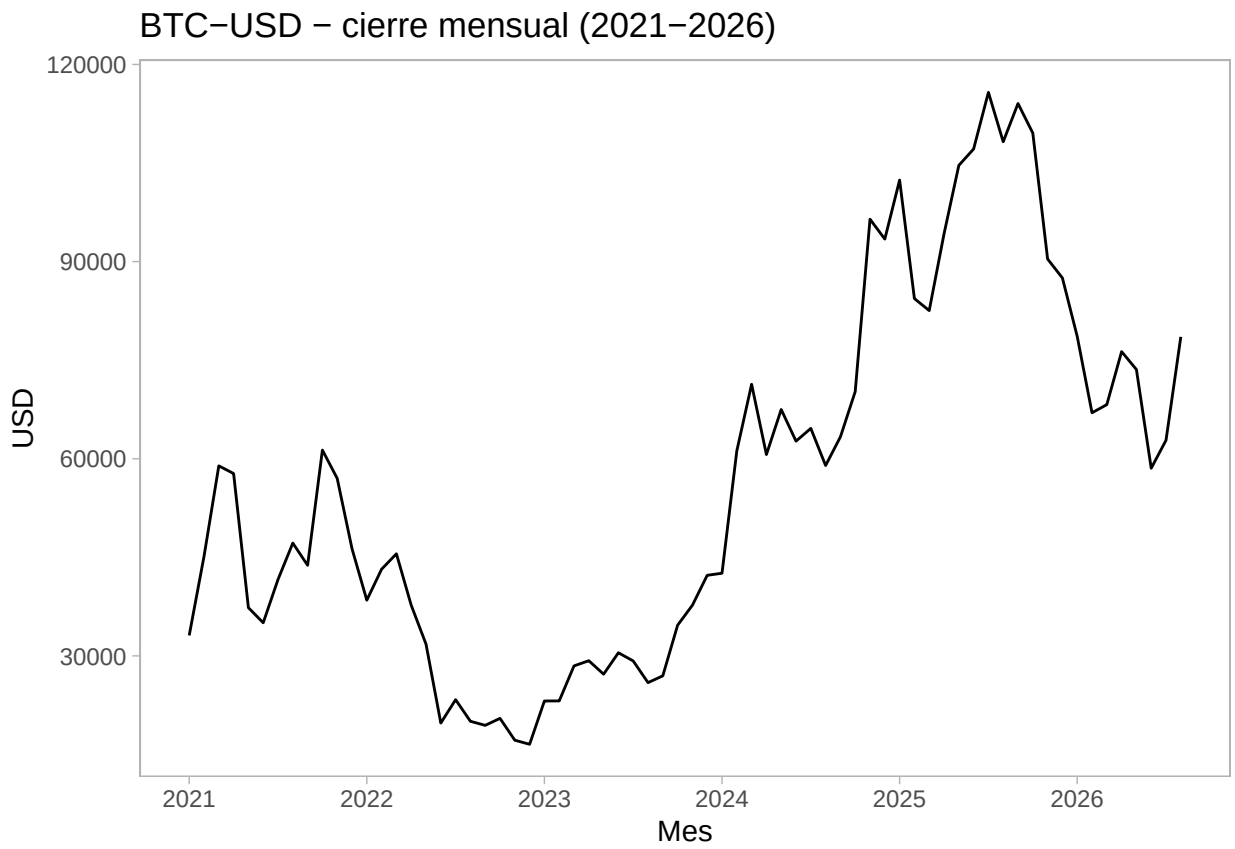
colcap_daily <- read.csv("data/colcap_daily.csv")
colcap_daily$date <- as.Date(colcap_daily$date)
colcap_xts <- xts(colcap_daily$colcap, order.by = colcap_daily$date)
colcap_m <- to.monthly(colcap_xts, indexAt = "lastof", OHLC = FALSE)
colcap_m <- window(colcap_m, start = as.Date("2021-01-01"), end = as.Date("2026-08-31"))
colcap_ts <- ts(as.numeric(colcap_m), start = c(2021, 1), frequency = 12)

btc_daily <- getSymbols("BTC-USD", src = "yahoo", from = "2020-12-01", to = "2026-09-01", auto.assign =
btc_m <- to.monthly(btc_daily, indexAt = "lastof", OHLC = FALSE)[, 4]
btc_m <- window(btc_m, start = as.Date("2021-01-01"), end = as.Date("2026-08-31"))
btc_ts <- ts(as.numeric(btc_m), start = c(2021, 1), frequency = 12)

forecast::autoplot(colcap_ts) +
  labs(title = "COLCAP - cierre mensual (2021-2026)", x = "Mes", y = "Índice") +
  theme_light() + theme(panel.grid = element_blank())
```




```
forecast::autoplot(btc_ts) +
  labs(title = "BTC-USD - cierre mensual (2021-2026)", x = "Mes", y = "USD") +
  theme_light() + theme(panel.grid = element_blank())
```



Ambas series muestran una tendencia de largo plazo (creciente, con caídas marcadas en 2022 y recuperación posterior) y varianza que cambia con el nivel — señal visual, antes de cualquier prueba formal, de que ninguna de las dos es estacionaria en niveles.

Los retornos mensuales de cada serie se calculan como el log-retorno: $r_t = \log(P_t) - \log(P_{t-1})$.

```
colcap_ret <- diff(log(colcap_ts))
btc_ret <- diff(log(btc_ts))
```

2. Proceso de adecuación de las series para hacerlas estacionarias

Para cada serie se corren las tres pruebas de raíz unitaria (ADF, Phillips-Perron, KPSS) sobre los precios en niveles, y luego sobre la serie transformada. Sobre la serie transformada solo se reejecutan ADF y PP (que comparten la hipótesis nula de raíz unitaria); KPSS se corre únicamente en niveles, así que el resultado final se apoya en ADF y PP.

Advertencia sobre KPSS. En niveles, KPSS **no rechaza** la estacionariedad (p 0.10 en las tres especificaciones, tanto en COLCAP como en BTC-USD), lo que contradice a ADF y PP (que no rechazan la raíz unitaria, p entre 0.51 y 0.99). No es una incoherencia: las hipótesis nulas son opuestas (ADF/PP: H0 = raíz unitaria; KPSS: H0 = estacionariedad) y con solo 68 observaciones todas las pruebas tienen poca potencia; en particular KPSS tiende a no rechazar H0 en muestras pequeñas. Con la regla "2 de 3" las series en niveles se tratan como no estacionarias (además de que la tendencia y los quiebres son evidentes en el gráfico), pero se deja constancia de que el veredicto no es unánime.

COLCAP — precio. Sobre el nivel, ADF y PP no rechazan la raíz unitaria en ninguna especificación (p-valores entre 0.85 y 0.99) y KPSS no rechaza estacionariedad (ver la advertencia arriba); por la regla 2 de 3 se trata como no estacionaria. Se aplica logaritmo natural (para estabilizar la varianza) y una diferencia (para quitar la tendencia). Sobre la serie ya transformada y diferenciada, ADF y PP rechazan la raíz

unitaria con claridad (p 0.01 en casi todas las especificaciones) — **una sola diferenciación (d=1) sobre el logaritmo ya alcanza.**

```
aTSA::adf.test(colcap_ts)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag  ADF p.value
## [1,]   0 1.54   0.967
## [2,]   1 1.92   0.985
## [3,]   2 1.50   0.964
## [4,]   3 1.45   0.961
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 0.556   0.986
## [2,]   1 1.127   0.990
## [3,]   2 0.550   0.986
## [4,]   3 0.321   0.977
## Type 3: with drift and trend
##      lag    ADF p.value
## [1,]   0 -0.965   0.937
## [2,]   1 -0.561   0.976
## [3,]   2 -0.822   0.955
## [4,]   3 -0.835   0.954
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(colcap_ts)
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    3 0.757   0.854
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3  1.38    0.99
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -2.35   0.957
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

```
aTSA::kpss.test(colcap_ts, lag.short = FALSE)
```

```
## KPSS Unit Root Test
## alternative: nonstationary
##
## Type 1: no drift no trend
##  lag  stat p.value
##    6 0.406     0.1
```

```
## -----
##  Type 2: with drift no trend
##  lag  stat p.value
##    6 0.339     0.1
## -----
##  Type 1: with drift and trend
##  lag  stat p.value
##    6 0.118     0.1
## -----------
## Note: p.value = 0.01 means p.value <= 0.01
##     : p.value = 0.10 means p.value >= 0.10
```

```
aTSA::adf.test(diff(log(colcap_ts)))
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
## [1,]   0 -8.82    0.01
## [2,]   1 -4.51    0.01
## [3,]   2 -3.35    0.01
## [4,]   3 -3.93    0.01
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -9.00    0.01
## [2,]   1 -4.68    0.01
## [3,]   2 -3.57    0.01
## [4,]   3 -4.27    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -9.40  0.0100
## [2,]   1 -4.95  0.0100
## [3,]   2 -3.74  0.0284
## [4,]   3 -4.48  0.0100
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(diff(log(colcap_ts)))
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    3   -83    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3 -83.6    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -85.5    0.01
## ---------------
```

```
## Note: p-value = 0.01 means p.value <= 0.01
```

COLCAP — retorno. El retorno mensual, por construcción, ya es la serie de precios transformada y diferenciada del punto anterior — las pruebas sobre el retorno dan el mismo resultado: estacionaria sin necesidad de diferenciar de nuevo (d=0).

```
aTSA::adf.test(colcap_ret)
```

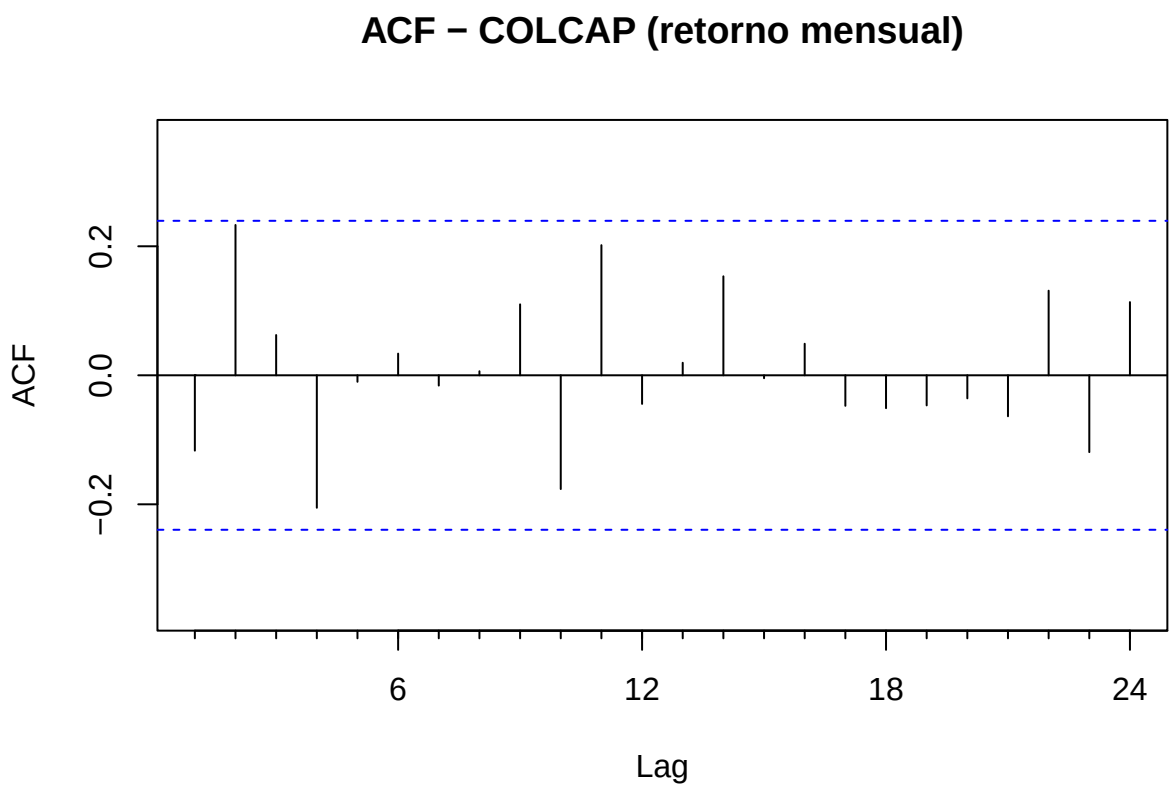
```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
## [1,]   0 -8.82    0.01
## [2,]   1 -4.51    0.01
## [3,]   2 -3.35    0.01
## [4,]   3 -3.93    0.01
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -9.00    0.01
## [2,]   1 -4.68    0.01
## [3,]   2 -3.57    0.01
## [4,]   3 -4.27    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -9.40  0.0100
## [2,]   1 -4.95  0.0100
## [3,]   2 -3.74  0.0284
## [4,]   3 -4.48  0.0100
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(colcap_ret)
```

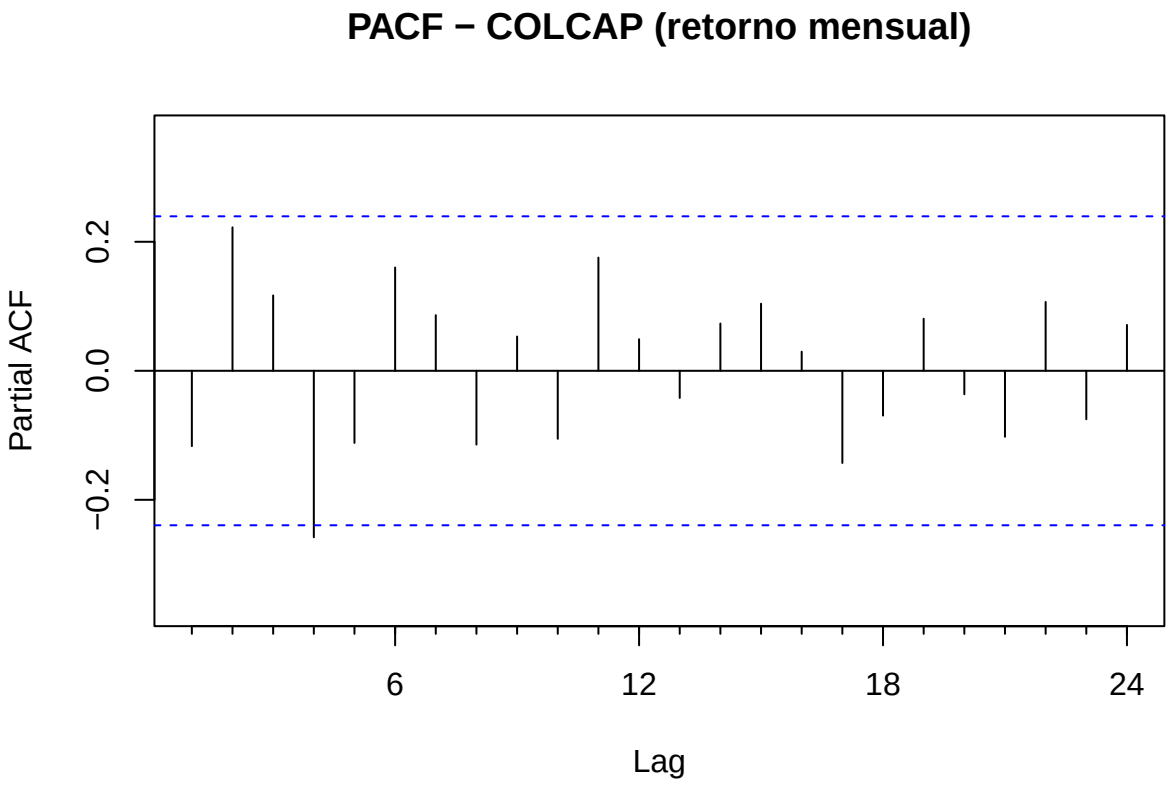
```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    3   -83    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3 -83.6    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -85.5    0.01
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

Como apoyo visual a las pruebas formales (y porque es la misma herramienta que se usa después para identificar p y q), se grafican ACF y PACF de esta misma serie estacionaria — es la misma serie que alimenta tanto el modelo de precio (vía log+diferencia) como el de retorno, así que este ACF/PACF sirve para ambos.

```r
forecast::Acf(colcap_ret, main = "ACF - COLCAP (retorno mensual)")
```




```r
forecast::Pacf(colcap_ret, main = "PACF - COLCAP (retorno mensual)")
```



Lectura del gráfico: con 67 observaciones, la banda de confianza al 95% es $\pm 1.96/\sqrt{67} \approx \pm 0.24$. Solo el rezago 4 de la PACF (-0.258) excede levemente esa banda; los rezagos 2 de la ACF (0.233) y de la PACF (0.222) quedan al borde sin superarla. Es una señal débil, consistente con que el modelo final (sección 3) resulte en un MA(2) donde solo uno de los dos coeficientes es significativo: no hay un patrón visual contundente, por eso se delega la decisión final a `auto.arima()` en vez de fijar p/q a ojo.

BTC-USD — precio. Sobre el nivel, ADF y PP no rechazan la raíz unitaria en ninguna especificación (p-valores todos > 0.5), mientras que KPSS tampoco rechaza estacionariedad (mismo desacuerdo descrito arriba); por la regla 2 de 3 se trata como no estacionaria. Igual que con COLCAP, log + una diferencia bastan para lograr estacionariedad (p 0.01 tras la transformación).

```
aTSA::adf.test(btc_ts)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag     ADF p.value
## [1,]   0 -0.0218   0.635
## [2,]   1 -0.1938   0.586
## [3,]   2 -0.2043   0.583
## [4,]   3 -0.0762   0.619
## Type 2: with drift no trend
##      lag    ADF p.value
## [1,]   0 -1.312   0.585
## [2,]   1 -1.308   0.586
## [3,]   2 -1.079   0.667
```

```
## [4,]   3 -0.941   0.715
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -1.77   0.663
## [2,]   1 -2.12   0.518
## [3,]   2 -2.13   0.513
## [4,]   3 -1.96   0.586
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(btc_ts)
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag    Z_rho p.value
##    3 -0.00993   0.685
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3 -3.58   0.583
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -6.85   0.656
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

```
aTSA::kpss.test(btc_ts, lag.short = FALSE)
```

```
## KPSS Unit Root Test
## alternative: nonstationary
##
## Type 1: no drift no trend
##  lag  stat p.value
##    6 0.284     0.1
## -----
##  Type 2: with drift no trend
##  lag  stat p.value
##    6 0.193     0.1
## -----
##  Type 1: with drift and trend
##  lag   stat p.value
##    6 0.0982     0.1
## -----------
## Note: p.value = 0.01 means p.value <= 0.01
##     : p.value = 0.10 means p.value >= 0.10
```

```
aTSA::adf.test(diff(log(btc_ts)))
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
```

```
## [1,]   0 -7.42    0.01
## [2,]   1 -6.34    0.01
## [3,]   2 -5.18    0.01
## [4,]   3 -3.75    0.01
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -7.38    0.01
## [2,]   1 -6.30    0.01
## [3,]   2 -5.15    0.01
## [4,]   3 -3.76    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -7.32  0.0100
## [2,]   1 -6.27  0.0100
## [3,]   2 -5.17  0.0100
## [4,]   3 -3.73  0.0291
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(diff(log(btc_ts)))
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    3 -54.7    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3 -54.9    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -54.5    0.01
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

BTC-USD — retorno. Mismo caso que COLCAP: el retorno ya es la serie transformada y diferenciada, estacionaria sin diferenciar de nuevo (d=0).

```
aTSA::adf.test(btc_ret)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
## [1,]   0 -7.42    0.01
## [2,]   1 -6.34    0.01
## [3,]   2 -5.18    0.01
## [4,]   3 -3.75    0.01
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -7.38    0.01
## [2,]   1 -6.30    0.01
```

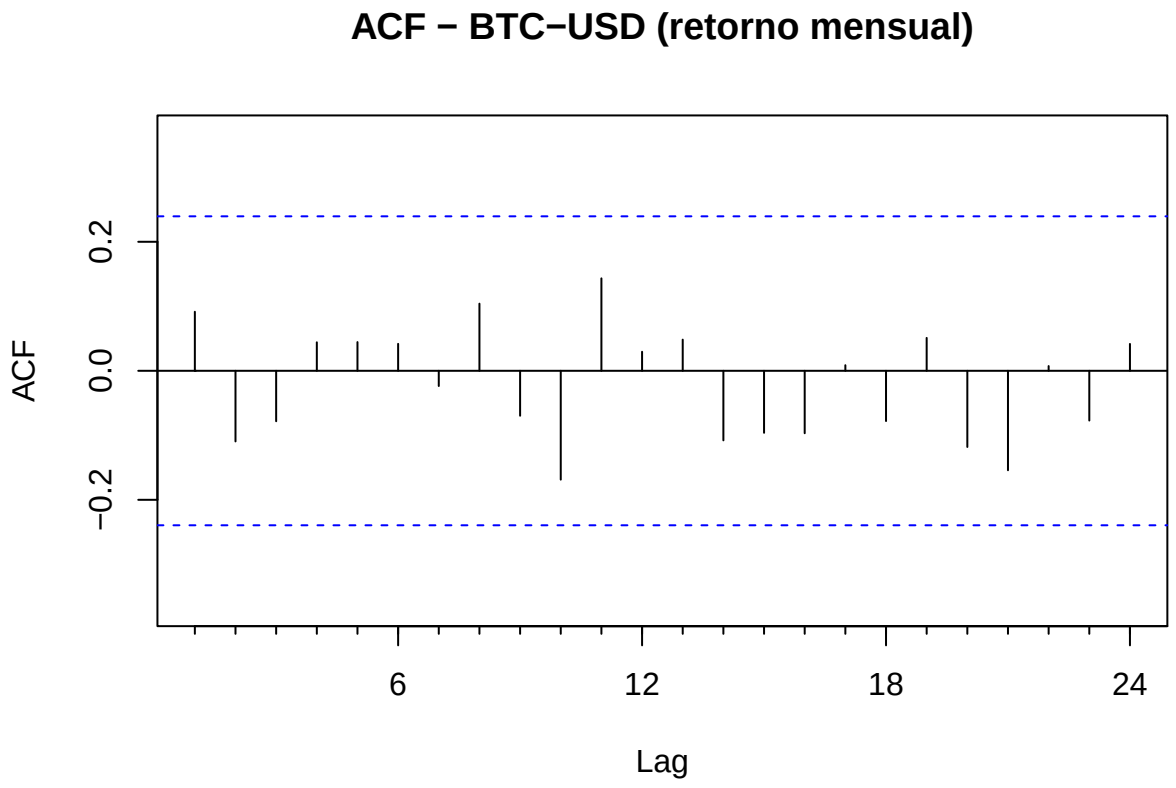
```
## [3,]   2 -5.15    0.01
## [4,]   3 -3.76    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -7.32  0.0100
## [2,]   1 -6.27  0.0100
## [3,]   2 -5.17  0.0100
## [4,]   3 -3.73  0.0291
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
aTSA::pp.test(btc_ret)
```

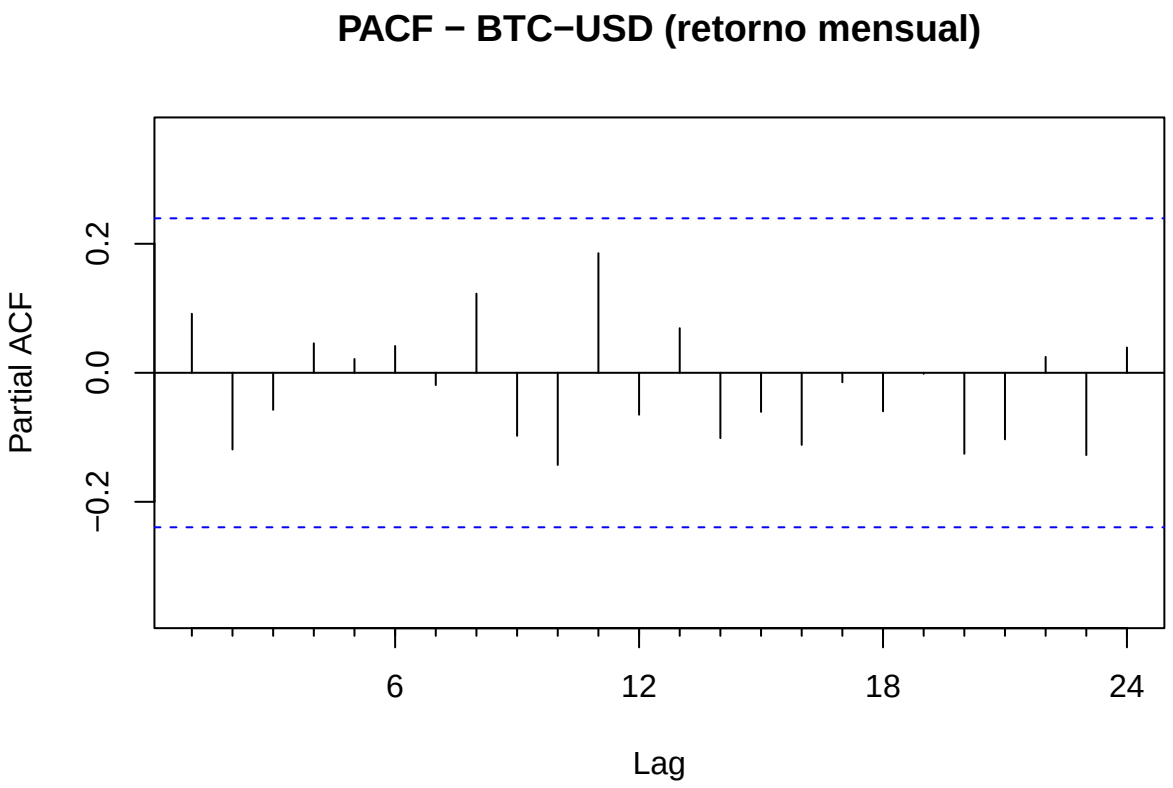
```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    3 -54.7    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    3 -54.9    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    3 -54.5    0.01
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

Igual que con COLCAP, se grafican ACF y PACF de la serie estacionaria (sirve tanto para el modelo de precio como el de retorno).

```
forecast::Acf(btc_ret, main = "ACF - BTC-USD (retorno mensual)")
```




```
forecast::Pacf(btc_ret, main = "PACF - BTC-USD (retorno mensual)")
```



Lectura del gráfico: con la misma banda (±0.24), **ningún rezago de la ACF ni de la PACF se sale de la banda** en ningún punto hasta el rezago 12 (el máximo en valor absoluto es 0.187) — a diferencia de COLCAP, ni siquiera hay un candidato "cercano". Es la confirmación visual de que el modelo final (sección 3) es ruido blanco puro: no hay ningún patrón que identificar.

3. Estimación

Para las 4 series (COLCAP precio, COLCAP retorno, BTC precio, BTC retorno) se usa `auto.arima()` con `stepwise=FALSE` (explora todas las combinaciones de p y q, no solo un atajo) sobre la serie ya adecuada del punto 2. Para las series de precio se pasa el precio original con `lambda=0`: `auto.arima()` aplica internamente el logaritmo (Box-Cox con lambda=0), ajusta sobre log(precio) y `forecast()` des-transforma el pronóstico (que resulta ser la mediana, no la media, salvo `biasadj=TRUE`).

3.1 COLCAP — precio

```
modelo_colcap_precio <- forecast::auto.arima(colcap_ts, stepwise = FALSE, lambda = 0)
modelo_colcap_precio
```

```
## Series: colcap_ts
## ARIMA(0,1,2)
## Box Cox transformation: lambda= 0
##
## Coefficients:
##           ma1     ma2
##       -0.1292  0.5339
## s.e.   0.0958  0.1398
```

```
##
## sigma^2 = 0.00268:  log likelihood = 104.01
## AIC=-202.02   AICc=-201.63   BIC=-195.4
```

```
lmtest::coeftest(modelo_colcap_precio)
```

```
##
## z test of coefficients:
##
##      Estimate Std. Error z value  Pr(>|z|)
## ma1 -0.129197   0.095805 -1.3485 0.1774837
## ma2  0.533871   0.139821  3.8182 0.0001344 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretación de los rezagos. El modelo encontrado es ARIMA(0,1,2): sin parte autorregresiva, una diferenciación, y 2 rezagos de media móvil. Es decir, el retorno mensual de COLCAP en el mes t no depende de sus propios valores pasados (p=0), pero sí del error/sorpresa de los 2 meses anteriores (q=2) — un shock inesperado en el mercado sigue "haciendo eco" hasta 2 meses después de ocurrir. La tabla de significancia (abajo) muestra que **ma2 es significativo (p<0.001) pero ma1 no lo es (p=0.177)** — el eco del shock de hace 2 meses pesa más que el de hace 1 mes.

3.2 COLCAP — retorno mensual

```
modelo_colcap_retorno <- forecast::auto.arima(colcap_ret, stepwise = FALSE)
modelo_colcap_retorno
```

```
## Series: colcap_ret
## ARIMA(0,0,2) with zero mean
##
## Coefficients:
##           ma1     ma2
##       -0.1292  0.5339
## s.e.   0.0958  0.1398
##
## sigma^2 = 0.002679:  log likelihood = 104.01
## AIC=-202.02   AICc=-201.63   BIC=-195.4
```

```
lmtest::coeftest(modelo_colcap_retorno)
```

```
##
## z test of coefficients:
##
##      Estimate Std. Error z value  Pr(>|z|)
## ma1 -0.129198   0.095804 -1.3486 0.1774775
## ma2  0.533873   0.139821  3.8183 0.0001344 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretación de los rezagos. Al modelar el retorno directamente (sin pasar por `lambda`), el resultado es ARIMA(0,0,2) con media cero — matemáticamente el mismo modelo del punto 3.1, solo que aplicado directamente sobre la serie ya diferenciada en vez de sobre el precio en niveles. Los coeficientes ma1/ma2 y su significancia son prácticamente idénticos.

3.3 BTC-USD — precio

```
modelo_btc_precio <- forecast::auto.arima(btc_ts, stepwise = FALSE, lambda = 0)
modelo_btc_precio
```

```
## Series: btc_ts
## ARIMA(0,1,0)
## Box Cox transformation: lambda= 0
##
## sigma^2 = 0.02818:  log likelihood = 24.5
## AIC=-47   AICc=-46.93   BIC=-44.79
```

Interpretación de los rezagos. El modelo encontrado es ARIMA(0,1,0): **sin parte autorregresiva, sin media móvil** — solo la diferenciación logarítmica. No hay ningún coeficiente que estimar: es un **camino aleatorio (random walk) puro**. En la práctica, esto dice que el mejor predictor del precio de Bitcoin el próximo mes es, literalmente, el precio de este mes — ni su propia historia (AR) ni los shocks recientes (MA) aportan información adicional una vez que se transformó la serie.

3.4 BTC-USD — retorno mensual

```
modelo_btc_retorno <- forecast::auto.arima(btc_ret, stepwise = FALSE)
modelo_btc_retorno
```

```
## Series: btc_ret
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 0.02818:  log likelihood = 24.5
## AIC=-47   AICc=-46.93   BIC=-44.79
```

Interpretación de los rezagos. ARIMA(0,0,0) con media cero — el retorno mensual de Bitcoin no tiene ninguna estructura identificable, es equivalente a **ruido blanco**. Ningún rezago, propio o de error, ayuda a explicarlo.

Tabla resumen de significancia de coeficientes:

Serie	Modelo	Coeficiente	Estimado	p-valor	¿Significativo?
COLCAP precio	ARIMA(0,1,2)	ma1	-0.129	0.177	No
COLCAP precio	ARIMA(0,1,2)	ma2	0.534	<0.001	Sí
COLCAP retorno	ARIMA(0,0,2)	ma1	-0.129	0.177	No
COLCAP retorno	ARIMA(0,0,2)	ma2	0.534	<0.001	Sí
BTC precio	ARIMA(0,1,0)	—	—	—	Sin coeficientes (random walk)
BTC retorno	ARIMA(0,0,0)	—	—	—	Sin coeficientes (ruido blanco)

4. Validación de los supuestos del modelo estimado

Para cada uno de los 4 modelos se revisan los mismos 4 supuestos sobre los residuales: media 0 (t-test), no autocorrelación (Ljung-Box sobre residuales), varianza constante (Ljung-Box sobre residuales al cuadrado + ARCH-LM) y normalidad (Jarque-Bera).

4.1 COLCAP — precio

```
res_colcap_precio <- modelo_colcap_precio$residuals
t.test(res_colcap_precio)
```

```
##
##  One Sample t-test
##
## data:  res_colcap_precio
## t = 1.0587, df = 67, p-value = 0.2935
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.005747234  0.018731268
## sample estimates:
##   mean of x
## 0.006492017
```

```
tabla.Box.Pierce(residuo = res_colcap_precio, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.008 0.92703
## 2        2     1.075 0.58411
## 3        3     1.121 0.77212
## 4        4     3.346 0.50163
## 5        5     3.469 0.62804
## 6        6     3.992 0.67779
## 7        7     4.000 0.77983
## 8        8     4.171 0.84135
## 9        9     4.171 0.89977
## 10      10     5.552 0.85137
## 11      11     8.538 0.66443
## 12      12     8.627 0.73444
```

```
tabla.Box.Pierce(residuo = res_colcap_precio^2, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.000 0.99572
## 2        2     0.211 0.89999
## 3        3     0.534 0.91130
## 4        4     0.735 0.94695
## 5        5     2.212 0.81908
## 6        6     2.414 0.87795
## 7        7     2.885 0.89545
## 8        8     2.987 0.93517
## 9        9     3.259 0.95314
## 10      10     3.309 0.97319
## 11      11     3.323 0.98573
## 12      12     3.362 0.99240
```

```
tabla.ARCH.LM(residuo = res_colcap_precio, max.lag = 12)
```

```
##    Rezagos ARCH LM-EstadÌstico p-valor
## 1        1       0.00002209052 0.99625
## 2        2       0.19784042871 0.90581
## 3        3       0.52373032025 0.91365
## 4        4       0.71505744645 0.94946
```

```
## 5        5       1.87254062146 0.86649
## 6        6       2.10144636888 0.91014
## 7        7       2.40725550709 0.93392
## 8        8       2.53327949370 0.96016
## 9        9       2.80887315163 0.97139
## 10      10       2.83816732309 0.98498
## 11      11       2.96433966338 0.99119
## 12      12       3.14698711588 0.99441
```

```
tseries::jarque.bera.test(res_colcap_precio)
```

```
##
##  Jarque Bera Test
##
## data:  res_colcap_precio
## X-squared = 7.4115, df = 2, p-value = 0.02458
```

Análisis: media 0 no se rechaza (p=0.29, cumple). Ljung-Box sobre los residuales no rechaza en ningún rezago hasta 12 (todos p>0.05) — no hay autocorrelación, cumple. Ljung-Box sobre residuales^2 y ARCH-LM tampoco rechazan en ningún rezago — varianza constante, cumple. **Jarque-Bera sí rechaza (p=0.025)** — los residuales no son normales. Como el supuesto que falla es normalidad y no varianza constante, no aplica el comentario de la nota del enunciado sobre ARCH/GARCH; simplemente se deja constancia de que este supuesto no se cumple del todo.

4.2 COLCAP — retorno mensual

```
res_colcap_retorno <- modelo_colcap_retorno$residuals
t.test(res_colcap_retorno)
```

```
##
##  One Sample t-test
##
## data:  res_colcap_retorno
## t = 1.0413, df = 66, p-value = 0.3015
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.005945367  0.018908118
## sample estimates:
##   mean of x
## 0.006481376
```

```
tabla.Box.Pierce(residuo = res_colcap_retorno, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.008 0.92756
## 2        2     1.058 0.58918
## 3        3     1.104 0.77622
## 4        4     3.295 0.50970
## 5        5     3.418 0.63583
## 6        6     3.933 0.68571
## 7        7     3.941 0.78657
## 8        8     4.110 0.84710
## 9        9     4.110 0.90403
## 10      10     5.472 0.85754
## 11      11     8.422 0.67511
## 12      12     8.511 0.74401
```

```r
tabla.Box.Pierce(residuo = res_colcap_retorno^2, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.001 0.97334
## 2        2     0.239 0.88754
## 3        3     0.554 0.90698
## 4        4     0.775 0.94181
## 5        5     2.291 0.80758
## 6        6     2.528 0.86529
## 7        7     3.032 0.88203
## 8        8     3.117 0.92681
## 9        9     3.430 0.94477
## 10      10     3.468 0.96817
## 11      11     3.492 0.98249
## 12      12     3.600 0.98963
```

```r
tabla.ARCH.LM(residuo = res_colcap_retorno, max.lag = 12)
```

```
##    Rezagos ARCH LM-EstadÌstico p-valor
## 1        1         0.001098434 0.97356
## 2        2         0.224975039 0.89361
## 3        3         0.506892487 0.91737
## 4        4         0.773218923 0.94200
## 5        5         1.856021351 0.86869
## 6        6         2.078233368 0.91236
## 7        7         2.392788998 0.93495
## 8        8         2.573386972 0.95822
## 9        9         2.837896241 0.97038
## 10      10         2.961304660 0.98232
## 11      11         3.040588988 0.99018
## 12      12         3.165426317 0.99426
```

```r
tseries::jarque.bera.test(res_colcap_retorno)
```

```
##
##  Jarque Bera Test
##
## data:  res_colcap_retorno
## X-squared = 6.763, df = 2, p-value = 0.034
```

Análisis: mismo patrón que 4.1 (es, en la práctica, el mismo modelo): media 0 cumple (p=0.30), no autocorrelación cumple, varianza constante cumple, y normalidad **no cumple** (p=0.034).

4.3 BTC-USD — precio

```r
res_btc_precio <- modelo_btc_precio$residuals
t.test(res_btc_precio)
```

```
##
##  One Sample t-test
##
## data:  res_btc_precio
## t = 0.63336, df = 67, p-value = 0.5287
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.02765765  0.05336819
```

```
## sample estimates:
##  mean of x
## 0.01285527
```

```
tabla.Box.Pierce(residuo = res_btc_precio, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.590 0.44237
## 2        2     1.460 0.48184
## 3        3     1.908 0.59173
## 4        4     2.057 0.72528
## 5        5     2.207 0.81978
## 6        6     2.340 0.88593
## 7        7     2.384 0.93558
## 8        8     3.245 0.91805
## 9        9     3.642 0.93337
## 10      10     5.975 0.81737
## 11      11     7.697 0.74018
## 12      12     7.772 0.80268
```

```
tabla.Box.Pierce(residuo = res_btc_precio^2, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     1.323 0.25005
## 2        2     2.242 0.32602
## 3        3     2.254 0.52145
## 4        4     3.585 0.46503
## 5        5     4.443 0.48754
## 6        6     4.796 0.57022
## 7        7     7.158 0.41262
## 8        8     8.366 0.39855
## 9        9     8.966 0.44043
## 10      10    10.414 0.40492
## 11      11    10.424 0.49270
## 12      12    10.849 0.54186
```

```
tabla.ARCH.LM(residuo = res_btc_precio, max.lag = 12)
```

```
##    Rezagos ARCH LM-EstadÌstico p-valor
## 1        1            1.258923 0.26186
## 2        2            1.690893 0.42937
## 3        3            2.323524 0.50803
## 4        4            3.882749 0.42211
## 5        5            4.080713 0.53785
## 6        6            3.873083 0.69385
## 7        7            5.480718 0.60151
## 8        8            7.902066 0.44309
## 9        9            8.821116 0.45395
## 10      10            7.905505 0.63807
## 11      11            7.783665 0.73254
## 12      12            7.491304 0.82351
```

```
tseries::jarque.bera.test(res_btc_precio)
```

```
##
##  Jarque Bera Test
##
```

```
## data:  res_btc_precio
## X-squared = 1.3099, df = 2, p-value = 0.5195
```

Análisis: los 4 supuestos se cumplen sin ninguna alerta — media 0 (p=0.53), no autocorrelación (todos los rezagos p>0.4), varianza constante (Ljung-Box² y ARCH-LM con p>0.25 en todos los rezagos), y normalidad (p=0.52, no se rechaza). Es el modelo mejor comportado de los 4, consistente con ser el más simple (random walk puro).

4.4 BTC-USD — retorno mensual

```
res_btc_retorno <- modelo_btc_retorno$residuals
t.test(res_btc_retorno)
```

```
##
##  One Sample t-test
##
## data:  res_btc_retorno
## t = 0.62575, df = 66, p-value = 0.5336
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.02824176  0.05402537
## sample estimates:
##  mean of x
## 0.01289181
```

```
tabla.Box.Pierce(residuo = res_btc_retorno, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     0.587 0.44361
## 2        2     1.440 0.48685
## 3        3     1.882 0.59734
## 4        4     2.025 0.73120
## 5        5     2.172 0.82480
## 6        6     2.305 0.88965
## 7        7     2.348 0.93813
## 8        8     3.196 0.92144
## 9        9     3.584 0.93663
## 10      10     5.892 0.82423
## 11      11     7.588 0.74971
## 12      12     7.661 0.81105
```

```
tabla.Box.Pierce(residuo = res_btc_retorno^2, max.lag = 12, type = "Ljung-Box")
```

```
##    Rezagos Ljung-Box p-valor
## 1        1     1.047 0.30615
## 2        2     2.127 0.34527
## 3        3     2.152 0.54148
## 4        4     2.865 0.58064
## 5        5     3.650 0.60080
## 6        6     3.994 0.67754
## 7        7     6.295 0.50573
## 8        8     7.417 0.49243
## 9        9     8.310 0.50326
## 10      10     9.856 0.45320
## 11      11     9.860 0.54299
## 12      12    10.265 0.59270
```

```r
tabla.ARCH.LM(residuo = res_btc_retorno, max.lag = 12)
```

```
##    Rezagos ARCH LM-EstadÌstico p-valor
## 1        1            1.031110 0.30990
## 2        2            2.305633 0.31575
## 3        3            2.349408 0.50312
## 4        4            3.286427 0.51109
## 5        5            3.824106 0.57501
## 6        6            3.958430 0.68230
## 7        7            5.393282 0.61209
## 8        8            7.677066 0.46563
## 9        9            6.639225 0.67462
## 10      10            7.783202 0.65000
## 11      11            7.458962 0.76079
## 12      12            7.325288 0.83539
```

```r
tseries::jarque.bera.test(res_btc_retorno)
```

```
##
##  Jarque Bera Test
##
## data:  res_btc_retorno
## X-squared = 1.145, df = 2, p-value = 0.5641
```

Análisis: igual que 4.3, los 4 supuestos se cumplen limpio (media 0 p=0.53, sin autocorrelación, varianza constante, normalidad p=0.56).

5. Pronósticos

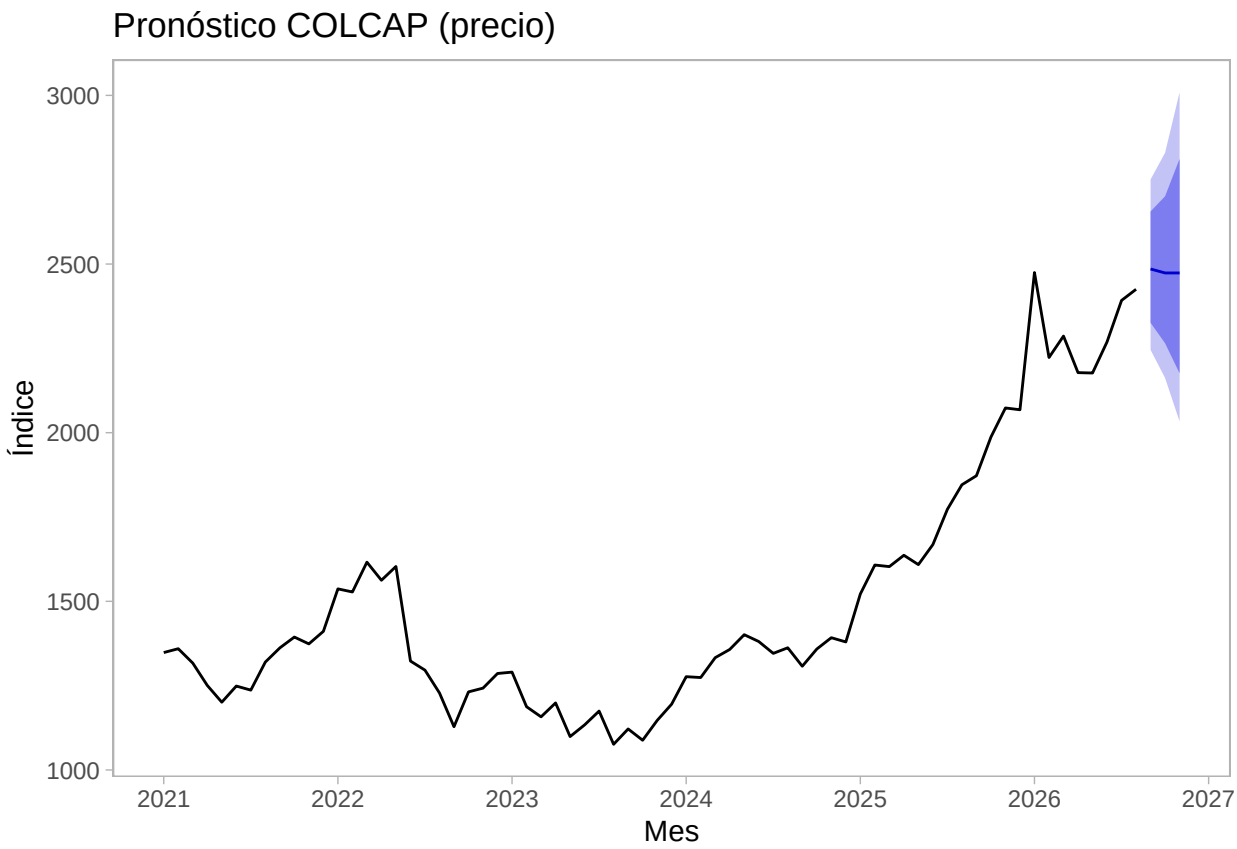
Se pronostican los 3 meses siguientes (septiembre-noviembre 2026) para cada una de las 4 series, con `forecast(modelo, h=3)`.

5.1 COLCAP — precio

```r
pron_colcap_precio <- forecast::forecast(modelo_colcap_precio, h = 3)
pron_colcap_precio
```

```
##          Point Forecast    Lo 80    Hi 80    Lo 95    Hi 95
## Sep 2026       2485.383 2325.856 2655.852 2245.595 2750.776
## Oct 2026       2473.536 2265.245 2700.979 2162.180 2829.728
## Nov 2026       2473.536 2176.033 2811.713 2033.315 3009.067
```

```r
forecast::autoplot(pron_colcap_precio) +
  labs(title = "Pronóstico COLCAP (precio)", x = "Mes", y = "Índice") +
  theme_light() + theme(panel.grid = element_blank())
```



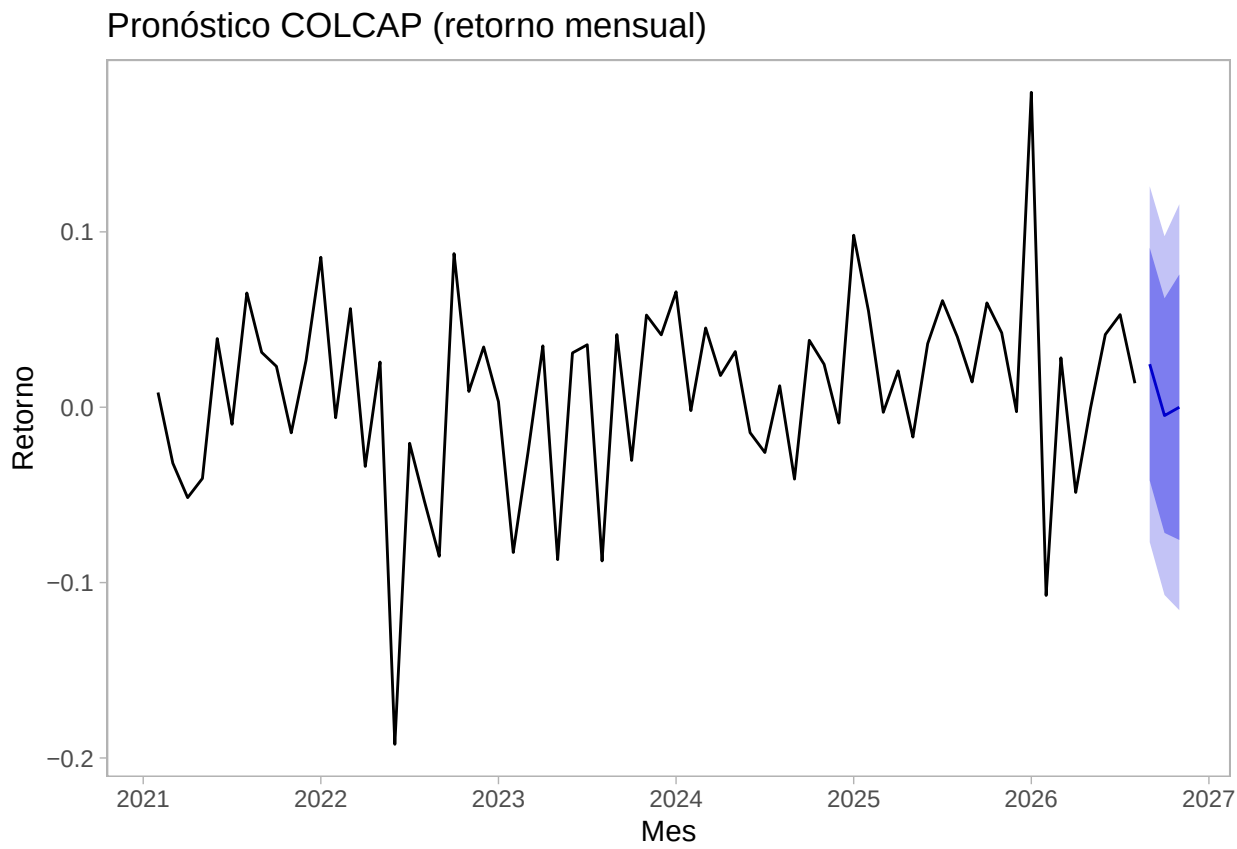
Interpretación del primer valor pronosticado: para septiembre de 2026 se espera un COLCAP de **2,485.4 puntos** (intervalo 95%: entre 2,245.6 y 2,750.8) — ligeramente por encima (+2.5%) del cierre real de agosto 2026 (2,425.1), coherente con el +2.46% de retorno de la sección 5.2. El pronóstico a un paso depende de los residuales recientes ponderados por ma1 (-0.129, no significativo) y ma2 (+0.534, que domina). Octubre y noviembre convergen al mismo valor (2,473.5), porque un modelo puramente MA(2) deja de "recordar" información después de 2 períodos.

5.2 COLCAP — retorno mensual

```
pron_colcap_retorno <- forecast::forecast(modelo_colcap_retorno, h = 3)
pron_colcap_retorno
```

```
##          Point Forecast       Lo 80      Hi 80       Lo 95      Hi 95
## Sep 2026    0.024562390 -0.04176631 0.09089109 -0.07687861 0.12600339
## Oct 2026   -0.004778214 -0.07165821 0.06210179 -0.10706235 0.09750592
## Nov 2026    0.000000000 -0.07567615 0.07567615 -0.11573668 0.11573668
```

```
forecast::autoplot(pron_colcap_retorno) +
  labs(title = "Pronóstico COLCAP (retorno mensual)", x = "Mes", y = "Retorno") +
  theme_light() + theme(panel.grid = element_blank())
```



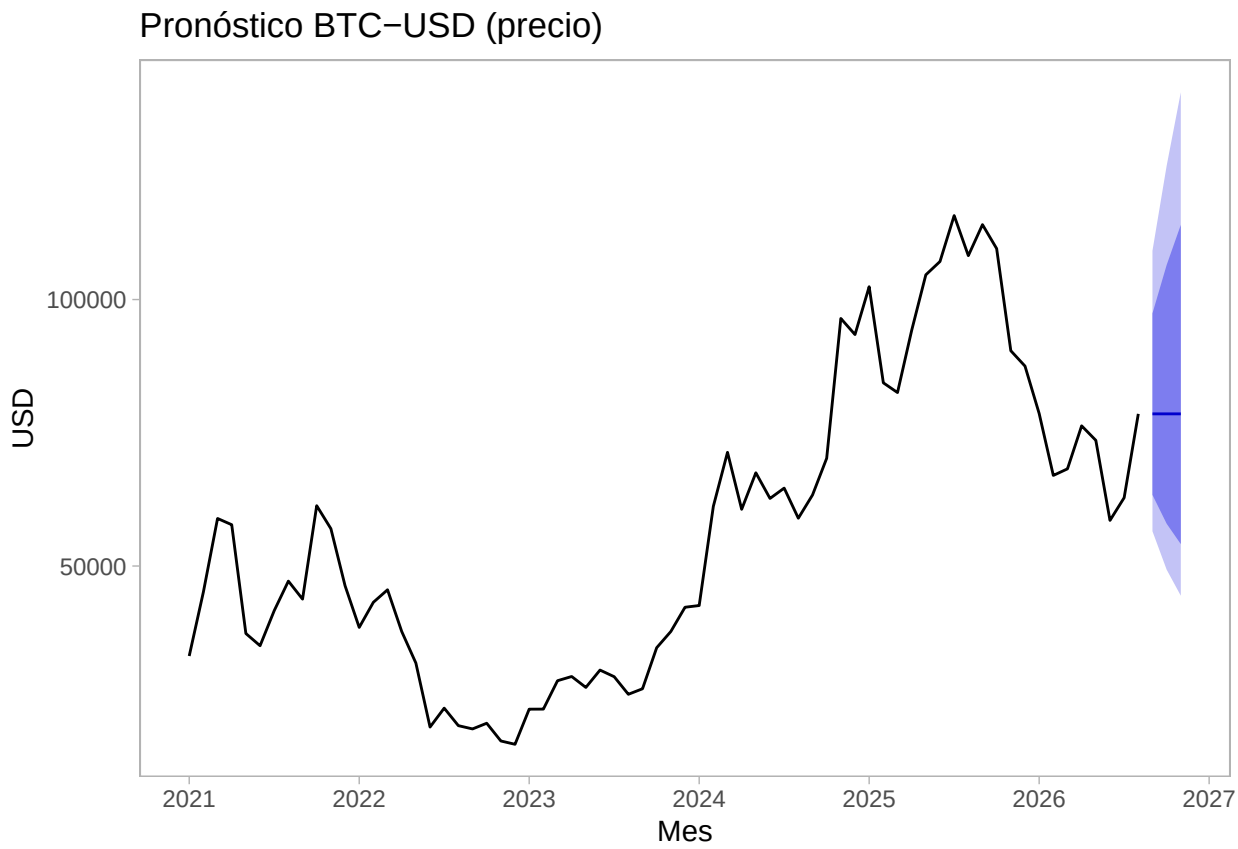
Interpretación del primer valor pronosticado: para septiembre de 2026 se espera un retorno mensual de **+2.46%** (intervalo 95%: entre -7.7% y +12.6%, un rango amplio típico de un activo volátil). Octubre pronostica -0.48% y noviembre 0.00% (una vez el modelo MA(2) agota su memoria, el pronóstico converge a la media incondicional).

5.3 BTC-USD — precio

```
pron_btc_precio <- forecast::forecast(modelo_btc_precio, h = 3)
pron_btc_precio
```

```
##          Point Forecast    Lo 80     Hi 80    Lo 95    Hi 95
## Sep 2026       78548.63 63343.96  97402.94 56525.52 109152.3
## Oct 2026       78548.63 57943.38 106481.33 49323.72 125089.7
## Nov 2026       78548.63 54113.69 114017.12 44426.16 138879.6
```

```
forecast::autoplot(pron_btc_precio) +
  labs(title = "Pronóstico BTC-USD (precio)", x = "Mes", y = "USD") +
  theme_light() + theme(panel.grid = element_blank())
```



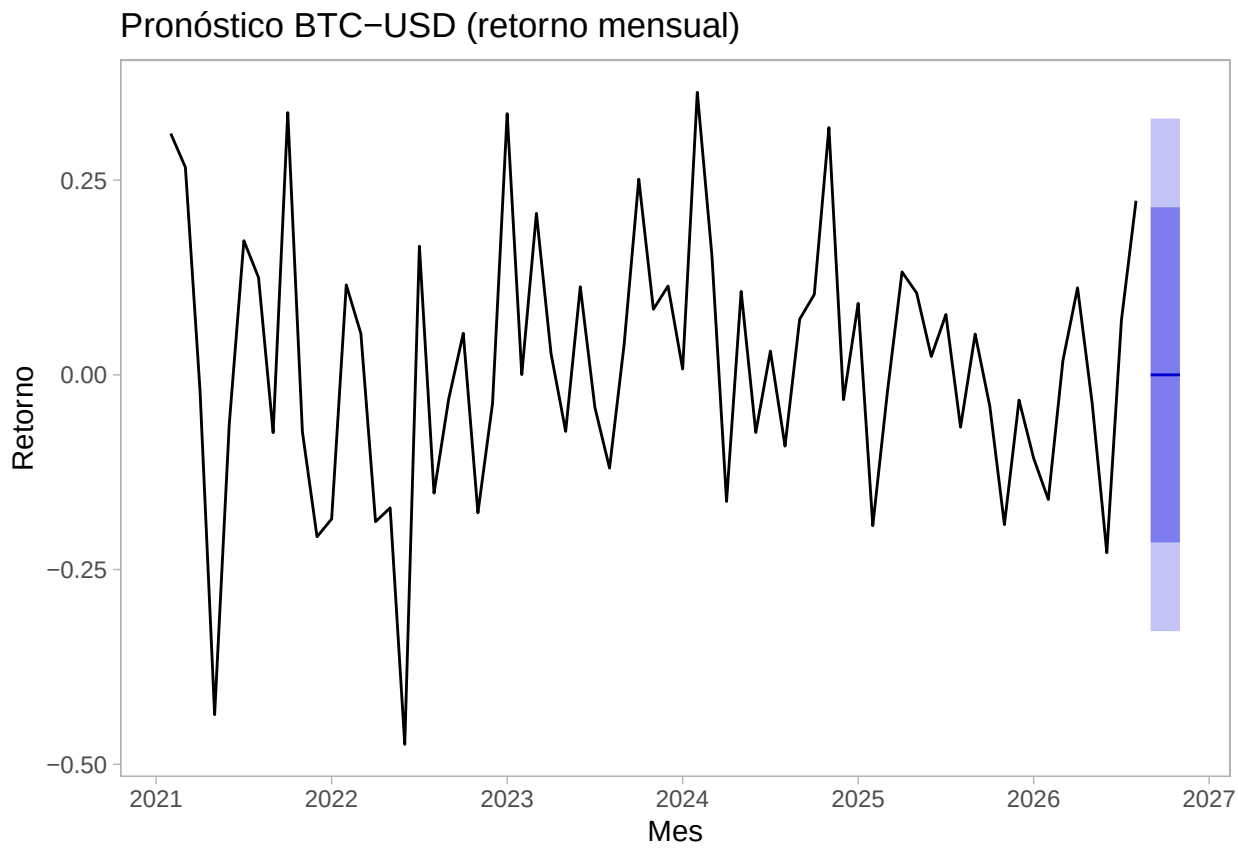
Interpretación del primer valor pronosticado: para septiembre de 2026 se espera un precio de **USD 78,548.6** — exactamente el último precio observado (agosto 2026), porque el modelo es un random walk sin deriva: la mejor predicción de mañana es el valor de hoy. Octubre y noviembre repiten el mismo punto central, con **intervalos de confianza que se ensanchan mes a mes** (al 95%, de [56,526; 109,152] en septiembre a [44,426; 138,880] en noviembre; al 80%, de [63,344; 97,403] en septiembre a [54,114; 114,017] en noviembre). Los intervalos no son simétricos alrededor del punto central: con `lambda=0` se construyen en escala logarítmica y luego se exponencian, por lo que son log-normales (más amplios hacia arriba que hacia abajo). Cada mes adicional acumula más incertidumbre sin que el modelo tenga ninguna señal direccional que la reduzca.

5.4 BTC-USD — retorno mensual

```
pron_btc_retorno <- forecast::forecast(modelo_btc_retorno, h = 3)
pron_btc_retorno
```

```
##          Point Forecast      Lo 80     Hi 80      Lo 95     Hi 95
## Sep 2026              0 -0.2151322 0.2151322 -0.3290164 0.3290164
## Oct 2026              0 -0.2151322 0.2151322 -0.3290164 0.3290164
## Nov 2026              0 -0.2151322 0.2151322 -0.3290164 0.3290164
```

```
forecast::autoplot(pron_btc_retorno) +
  labs(title = "Pronóstico BTC-USD (retorno mensual)", x = "Mes", y = "Retorno") +
  theme_light() + theme(panel.grid = element_blank())
```



Interpretación del primer valor pronosticado: para septiembre de 2026 se espera un retorno de **0.00%** (intervalo 95%: entre -32.9% y +32.9%) — el modelo de ruido blanco no tiene ninguna base para predecir dirección, así que el pronóstico puntual es simplemente "sin cambio esperado", con toda la incertidumbre reflejada en un intervalo muy amplio.

Conclusión general. COLCAP mostró cierta estructura lineal en la media en muestra (un MA(2) con el segundo rezago significativo; AIC -202.0 frente a -195.6 del ruido blanco), mientras que Bitcoin no mostró ninguna — ni en precio (random walk puro) ni en retorno (ruido blanco). Conviene matizar: (i) en COLCAP la evidencia es modesta, pues el Ljung-Box del retorno crudo no rechaza ($p = 0.09$ en el rezago 2 y 0.22 en el 12), y no se validó fuera de muestra ni se consideraron costos de transacción, así que llamarla "estructura explotable" sería excesivo; (ii) un resultado de ruido blanco con $n = 67$ solo indica ausencia de predictibilidad lineal en la media (consistente con eficiencia en sentido débil) y con poca potencia estadística: no demuestra eficiencia de mercado; (iii) atribuir la diferencia entre ambos activos a que Bitcoin es "más líquido y global" es una hipótesis plausible, pero esta muestra no la prueba.